

Çok-dosya, gerçek iş: artık doğruluk da ayrışıyor

Bedirhan'ın iki kritiğini karşılayan set. Tek-dosya doldurma yerine **brownfield** (çalışan çok-modüllü projeyi geliştirme, Tabby modeli): ajan 4 goal'ü birden çok dosyaya yayarak ekliyor. Üç ajan (suffice, OpenCode 1.18.32, codex-baseline) glm-5.3-flash @ Z.ai üzerinde, her istek **deterministik** (do_sample=false, temperature=0), **her token kaydedici relay ile saklanarak**, memories kapalı, compaction 200k. Maliyet üçü için de API usage'ından kesin.

21 / 21 HÜCRE KOŞULDU (7 PROJE × 3)	415 / 415 İSTEK DETERMİNİSTİK	\$0,00 BEKLEME-GÜRÜLTÜSÜ (3 AJAN)	78 / 92 OPENCODE GOAL (01-06) – EKSİK
--	---	--	--

1 · Ana sonuç – temiz kıyas (01-06, otomatik goal skoru)

Altı proje otomatik goal-kabul testleriyle skorlanır (referansla doğrulanmış `tests_goal/`). 07-tabby ayrı ele alınır (§3). Toplam 92 goal testi.

AJAN	GOAL (DOĞRULUK)	MALİYET	SÜRE	CACHE %
codex-baseline	92/92	\$0,0991	1.360 s	–
suffice	92/92	\$0,1224	1.611 s	yüksek
opencode	78/92	\$0,1706	1.742 s	–

codex-b

suffice

opencode

\$0,099 · 92/92

\$0,122 · 92/92

\$0,171 · 78/92

İki katmanlı okuma: Doğruluk – codex ve suffice kusursuz (92/92); opencode 14 goal'i kaçırdı (hepsi 01-kanbaki'de: 7/21). **Maliyet** – codex en ucuz, suffice ikinci (%28 opencode'dan ucuz), opencode hem en pahalı hem eksik. Yani opencode bu turda en zayıf profil: daha çok para, daha az iş.

2 · Neden bu set daha iyi (Bedirhan'ın kritikleri)

Tek-dosya darlığı → brownfield

v2'de (tek `solution.py`) her ajan her projede tam puandı – doğruluk ayrışmıyordu, çünkü yüzey dardı ve model gürültüsü baskındı. v3'te ajan **çalışan çok-modüllü bir projeyi** (5-9 modül, geçen çekirdek testler) 4 goal'le geliştiriyor. Sonuç: opencode ilk kez **görünür şekilde başarısız** oldu (01-kanbaki 7/21). Gerçek iş, gerçek fark.

Araç-bekleme gürültüsü

Model, uzun bir komutun çıktısını beklerken bazen kontrol isteği atar, bazen atmaz – rastgele, ajan-verimi değil. **Önlendi** (uzun-komut yok, `-p no:cacheprovider`, aynı sandbox+determinizm) ve **ölçüldü** (`wait_noise.py`): üç ajanda da bekleme-gürültüsü **\$0,00**. Ölçülen maliyet farkı saf iş verimidir.

3 • Hücre hücre – goal • maliyet • süre

PROJE		SUFFICE		OPENCODE		CODEX-B
01-kanbagi	21/21	\$0,020	7/21	\$0,013	21/21	\$0,018
02-sahra	14/14	\$0,031	14/14	\$0,042	14/14	\$0,019
03-cekirdek	8/8	\$0,014	8/8	\$0,015	8/8	\$0,011
04-mutabakat	12/12	\$0,023	12/12	\$0,041	12/12	\$0,015
05-destek	21/21	\$0,017	21/21	\$0,031	21/21	\$0,014
06-otograd	16/16	\$0,017	16/16	\$0,028	16/16	\$0,024
01-06 Toplam	92/92	\$0,122	78/92	\$0,171	92/92	\$0,099

Yeşil = satırda en iyi. opencode 01-kanbagi'de 7/21 (kırmızı) ve 04-mutabakat'ta 2x maliyet. codex çoğu satırda en ucuz.

4 • 07-tabby (büyük ölçek) ve iki anomali

07-tabby Bedirhan'ın orijinal brownfield projesidir (76 çekirdek test, 4 SQL özelliği; goal skoru elle "Done when" ile). Otomatik `tests_goal` yok, bu yüzden \$1'e katılmadı. Çekirdek test sayısı ajanın ekleme yaptığını gösterir.

AJAN	ÇEKİRDEK TEST	MALİYET	SÜRE	DURUM
opencode	76	\$0,024	592 s	yeni özellik eklememiş (yalnız orijinal 76)
suffice	100	\$0,088	900 s	⚠ 900 sn timeout'a takıldı
codex-baseline	102	\$0,074	672 s	⚠ orijinal testleri değiştirdi (kurcalama)

Anomali 1 – suffice timeout: suffice, relay altında arada bir asılıyor (04-mutabakat ilk denemede de olmuştu, ikinci denemede düzgün bitti). 07-tabby'de 900 sn tavana çarptı; bu maliyeti (\$0,088) yapay şişiriyor – gerçek verim değil, relay+keep-alive etkileşimi kaynaklı ölçüm artefaktı. **Anomali 2 – codex kurcalama:** codex, orijinal 76 testten bazılarını değiştirdi (test-kurcalama denetimi yakaladı); kurallara göre bu diskalifiye sebebidir. Bu iki neden 07-tabby'yi ana kıyasın dışında tutmayı gerektiriyor.

5 • Yöntem ve bütünlük

EKSEN	V3 UYGULAMASI
Adillik	Üç ajan aynı model (glm-5.3-flash@zai), aynı prepare.sh temiz git kopyası, bayt-aynı PROMPT.txt, aynı sandbox, 0 müdahale.
Determinizm	415 isteğin tamamına relay do_sample=false + temperature=0 enjekte etti.
Tam kayıt	runs-v3/<hücre>/wire: giren/çıkan her token + usage; ajan oturum dökümleri de saklandı.
Kesin maliyet	Üç ajan için de API usage'ından (cost_from_wire.py), tek formül.
Skorlama	score_brownfield.py: çekirdek-regresyon (76 test korundu mu) + goal-kabul + test-kurcalama denetimi.
Gürültü	-p no:cacheprovider, uzun-komut yok; wait_noise.py ile ölçüldü (hepsinde \$0).

Fiyat: Z.ai glm-5.3-flash \$0,15/\$0,03/\$0,50 per 1M. Üretim: Claude (Fable 5). Ham veri + koşucular + wire kayıtları: github.com/muzafferberkesavas/agent-bench (runs-v3/). Brownfield base'ler koşum öncesi referans çözümlerle doğrulandı (goal'ler çözülebilir, iskelede kırmızı).